EX NO :4A **DATE** :	**RANDOM FOREST REGRESSION**

AIM:

To implement the Random Forest Regression

PROCEDURE:

- Load the dataset containing features.
- Split the dataset into training and testing sets.
- Train a Random Forest Regression model using the training data.
- Evaluate the model's performance using the testing data.
- Optionally, analyze coefficients and interpret their significance.

CODE:

 # Importing libraries

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score , mean_squared_error
from sklearn import tree
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import r2_score
```

#Importing Diabetes Dataset

```
df=pd.read_csv('/content/drive/MyDrive/mlt/diabetes.csv')
```

#Checking for null Values

```
df.isna().sum()
```

OUTPUT:

	0
Glucose	0
BloodPressure	0
SkinThickness	0
Insulin	0
BMI	0
DiabetesPedigreeFunction	0
Age	0
Pregnancies	0

dtype: int64

```
df.head()
```

OUTPUT:

	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Pregnancies
0	148	72	35	0	33.6	0.627	50	6
1	85	66	29	0	26.6	0.351	31	1
2	183	64	0	0	23.3	0.672	32	8
3	89	66	23	94	28.1	0.167	21	1
4	137	40	35	168	43.1	2.288	33	0

#x and y

```
x=df.iloc[:,:-1]
y=df.iloc[:,-1]
```

#Splitting training and testing data

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2)
x_train
```

#Label encoder

```
le = LabelEncoder()
d.iloc[:,-1]=le.fit_transform(d.iloc[:,-1])
d.head()
```

OUTPUT:

	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
603	150	78	29	126	35.2	0.692	54
118	97	60	23	0	28.2	0.443	22
247	165	90	33	680	52.3	0.427	23
157	109	56	21	135	25.2	0.833	23
468	120	0	0	0	30.0	0.183	38

#Regressor

```
regressor=RandomForestRegressor(n_estimators=5,max_depth=4)
regressor.fit(x_train,y_train)
```

OUTPUT:

```
RandomForestRegressor
RandomForestRegressor(max_depth=4, n_estimators=5)
```

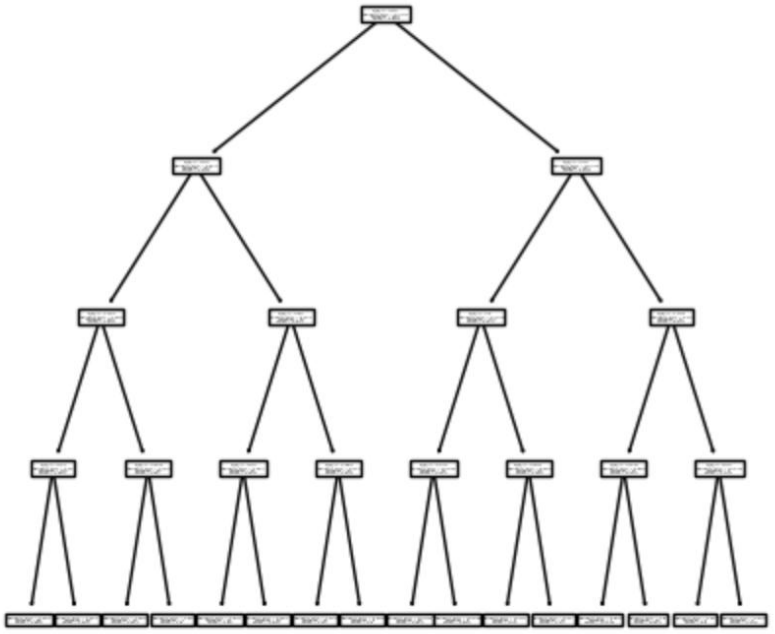
#Predict

```
y_pred=regressor.predict(x_test)
```

#Tree

```
fig=plt.figure(figsize=(5,5))
tree.plot_tree(regressor.estimators_[3])
plt.show()
```

OUTPUT:



#Mean squared error

```
mse=mean_squared_error(y_test,y_pred)
print(“Mean Squared Error:”, mse)
```

OUTPUT:

```
Mean Squared Error: 5.483664680366834
```

 # R-squared

```
r2 = r2_score(y_test, y_pred)
print("R2 Score:",r2)
```

OUTPUT:

```
R2 Score: 0.3611864829240289
```

#accuracy score

```
acc=regressor.score(x_test,y_test)
print("Accuracy Score:",acc*100,"%")
```

OUTPUT:

```
Accuracy Score: 41.21422624641554 %
```

Result:

Thus, implemented Random Forest Regressor successfully and executed.

EX NO :4B DATE :	IMPLEMENTATION OF RANDOM FOREST CLASSIFICATION

AIM:

To implement the Random Forest Classification.

PROCEDURE:

- Load the dataset containing features.
- Split the dataset into training and testing sets.
- Train a Random Forest Classifier model using the training data.
- Evaluate the model's performance using the testing data.
- Optionally, analyze coefficients and interpret their significance.

CODE:

#Importing libraries and dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score , mean_squared_error
from sklearn import tree

d=pd.read_csv('/content/drive/MyDrive/mlt/DiabetesClassify.csv')
d.head()
```

OUTPUT:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

#Splitting the dataset

```
x=d.iloc[:,:-1]
y=d.iloc[:,-1]
```

#Train test split

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
x_train.head()
```

OUTPUT:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
60	2	84	0	0	0	0.0	0.304	21
618	9	112	82	24	0	28.2	1.282	50
346	1	139	46	19	83	28.7	0.654	22
294	0	161	50	0	0	21.9	0.254	65
231	6	134	80	37	370	46.2	0.238	46

 # Random forest tree

```
clf=RandomForestClassifier(n_estimators=5,max_depth=4)
clf.fit(x_train,y_train)
```

OUTPUT:

```
RandomForestClassifier
RandomForestClassifier(max_depth=4, n_estimators=5)
```

#Predict

```
y_pred=clf.predict(x_test).astype(int)
y_pred
```

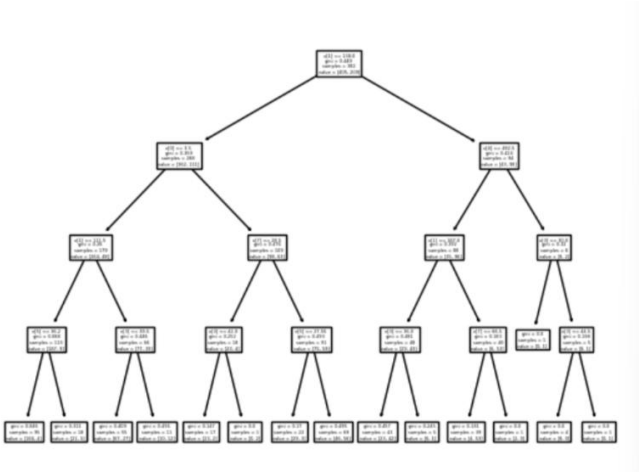
OUTPUT:

```
array([1, 0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0,
       0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0,
       0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 1, 0, 1,
       0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1,
       0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1,
       0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1])
```

#Plotting

```
fig=plt.figure()
tree.plot_tree(clf.estimators_[4])
plt.show()
```

OUTPUT:



#accuracy score

```
acc=clf.score(x_test,y_test)
print("Accuracy Score:",acc*100,"%")
```

OUTPUT:

```
Accuracy Score: 73.37662337662337 %
```

#Mean squared error

```
mse=mean_squared_error(y_test,y_pred)
print("Mean Squared Error:",mse)
```

OUTPUT:

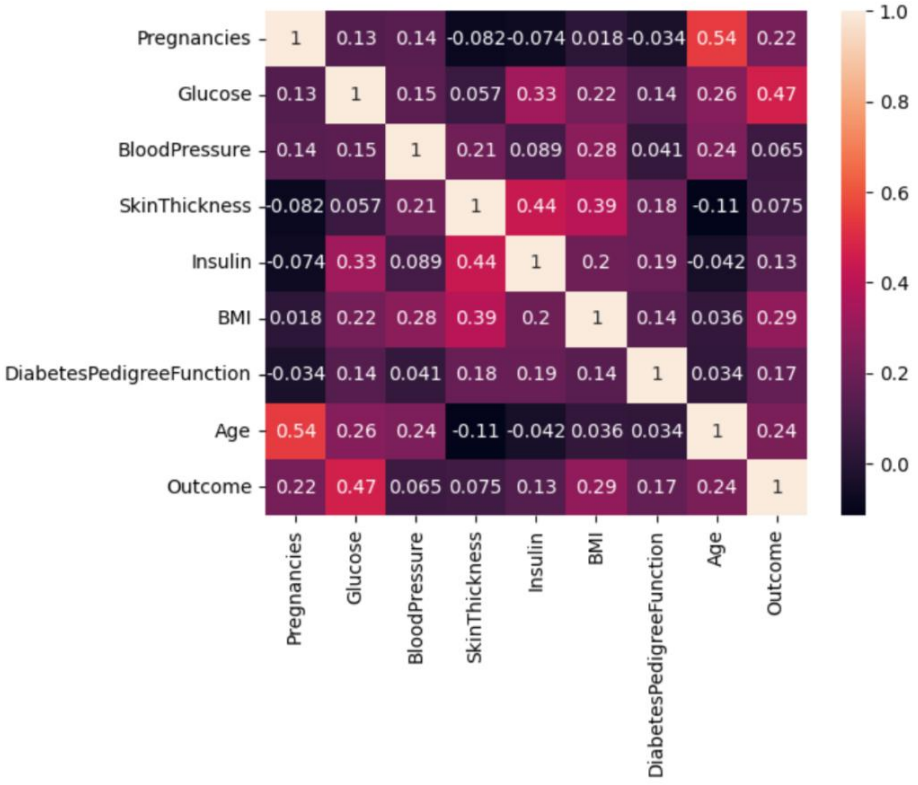
```
Mean Squared Error: 0.2662337662337662
```

#Heatmap

```
import seaborn as sns
sns.heatmap(d.corr(),annot=True)
```

OUTPUT:

```
<Axes: >
```



#confusion matrix

```
from sklearn.metrics import confusion_matrix
cm=confusion_matrix(y_test,y_pred)
cm
```

OUTPUT:

```
array([[81, 18],
       [23, 32]])
```

Result:

Thus implemented Random Forest Classifier successfully and executed.

EX NO : 5A **DATE** :	**IMPLEMENTATION OF SUPPORT VECTOR REGRESSION**

AIM:

To implement the support vector regression.

PROCEDURE:

- Load the dataset containing features.
- Split the dataset into training and testing sets.
- Train a SVR model using the training data.
- Evaluate the model's performance using the testing data.
- Optionally, analyze coefficients and interpret their significance.

CODE:

#Importing libraries and dataset

```
import pandas as pd
df = pd.read_csv('/content/drive/MyDrive/mlt/salary - salary.csv')
df.head()
```

OUTPUT:

	YearsExperience	Salary
0	1.1	39343.0
1	1.3	NaN
2	1.5	NaN
3	2.0	43525.0
4	2.2	39891.0

#Check for missing values in the dataset.

```
df.isna().sum()
```

OUTPUT:

	0
YearsExperience	0
Salary	2

dtype: int64

#Filling missing data by taking mean value

```
df['Salary'].fillna(df['Salary'].mean(),inplace=True)
df.isna().sum()
```

OUTPUT:

	0
YearsExperience	0
Salary	0

dtype: int64

```
x=df['YearsExperience'].values
y=df['Salary'].values
```

#Split the dataset for training and testing

```
from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=0)
```

#Importing SVR

```
from sklearn.svm import SVR
model=SVR(kernel="linear")
model.fit(x_train.reshape(-1,1),y_train)
```

OUTPUT:

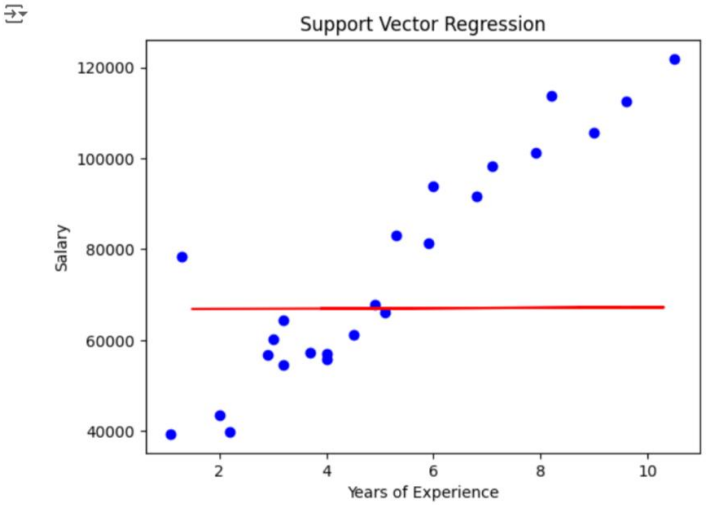
```
SVR
SVR(kernel='linear')
```

```
y_pred=model.predict(x_test.reshape(-1,1))
x_pred=model.predict(x_train.reshape(-1,1))
```

#Graph for training data

```
import matplotlib.pyplot as plt
plt.scatter(x_train,y_train,color='blue')
plt.plot(x_test,y_pred,color='red')
plt.title("Support Vector Regression")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.show()
```

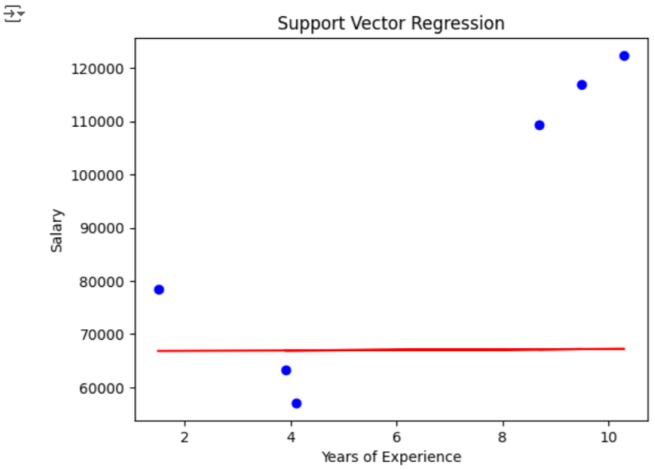
OUTPUT:



#Graph for Testing

```
import matplotlib.pyplot as plt
plt.scatter(x_test,y_test,color='blue')
plt.plot(x_test,y_pred,color='red')
plt.title("Support Vector Regression")
plt.xlabel("Years of Experience")
plt.ylabel("Salary")
plt.show()
```

OUTPUT:



#Mean Squared Error

```
from sklearn.metrics import mean_squared_error
mse=mean_squared_error(y_test,y_pred)
print("Mean Square Error:",mse)
```

OUTPUT:

```
Mean Square Error: 1259545054.363136
```

Result:

Thus implemented Support Vector Regression successfully and executed.

EX NO : 5B DATE :	IMPLEMENTATION OF SUPPORT VECTOR CLASSIFICATION

AIM:

To implement the support vector classification for image classification.

PROCEDURE:

- Load the dataset containing features.
- Split the dataset into training and testing sets.
- Train a SVC model using the training data.
- Evaluate the model's performance using the testing data.
- Optionally, analyze coefficients and interpret their significance.

CODE:

#Importing libraries and dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

df= pd.read_csv('/content/drive/MyDrive/mlt/iris.csv')
df.head()
```

OUTPUT:

	sepal length in cm	sepal width in cm	petal length in cm	petal width in cm	class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

#Columns

```
df.columns
```

OUTPUT:

```
Index(['sepal length in cm', 'sepal width in cm', 'petal length in cm',
       'petal width in cm', 'class'],
      dtype='object')
```

#Renaming the column names

```
df.rename(columns = {'sepal length in cm':'SepalLength', 'sepal width in cm':'SepalWidth','petal length in cm':'PetalLength','petal width in cm':'PetalWidth'}, inplace = True)
df.columns
```

OUTPUT:

```
Index(['SepalLength', 'SepalWidth', 'PetalLength', 'PetalWidth', 'class'], dtype='object')
```

#After renaming column names.

```
df.head()
```

OUTPUT:

	SepalLength	SepalWidth	PetalLength	PetalWidth	class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

```
x=df[['PetalLength','PetalWidth']]
species_to_num = { 'Iris-setosa'      : 0,
                   'Iris-versicolor' : 1,
                   'Iris-virginica'    : 2 }
df['tmp'] = df['class'].map(species_to_num)
y=df['tmp']
```

#Splitting the dataset into training and testing dataset.

```
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.2, random_state=0)
x_train.head()
```

OUTPUT:

	PetalLength	PetalWidth
137	5.5	1.8
84	4.5	1.5
27	1.5	0.2
127	4.9	1.8
132	5.6	2.2

```
x_test
```

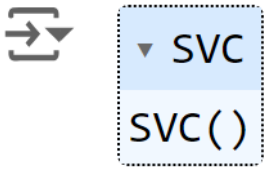
OUTPUT:

	PetalLength	PetalWidth
114	5.1	2.4
62	4.0	1.0
33	1.4	0.2
107	6.3	1.8
7	1.5	0.2

#Importing SVC

```
from sklearn.svm import SVC
svc_class = SVC(kernel='rbf')
svc_class.fit(x_train, y_train)
```

OUTPUT:

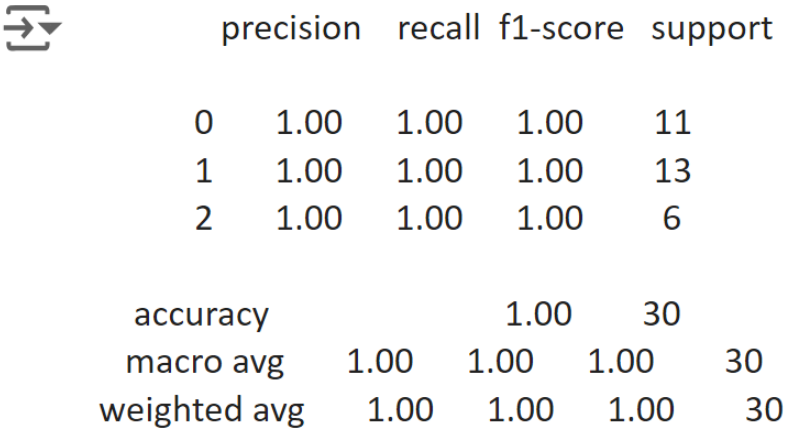



```
y_pred = svc_class.predict(x_test)
x_pred = svc_class.predict(x_train)
```

#Classification Report

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```

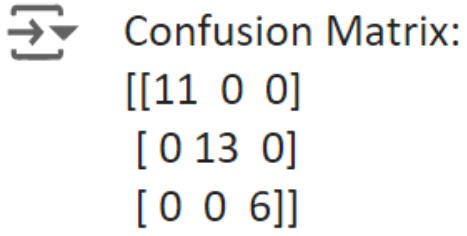
OUTPUT:



#Confusion Matrix

```
from sklearn import metrics
metrics.confusion_matrix(y_test, y_pred)
```

OUTPUT:



#Accuracy

```
accuracy = metrics.accuracy_score(y_test, y_pred)
print("Accuracy: ",accuracy)
```

OUTPUT:

Accuracy: 1.0

#Recall

```
recall = metrics.recall_score(y_test, y_pred, average='micro')
print("Recall: ",recall)
```

OUTPUT:

Recall: 1.0

 # support

```
from sklearn.metrics import precision_recall_fscore_support
precision, recall, f1_score, support = precision_recall_fscore_support(y_test, y_pred)
print("Support:", support[0])
```

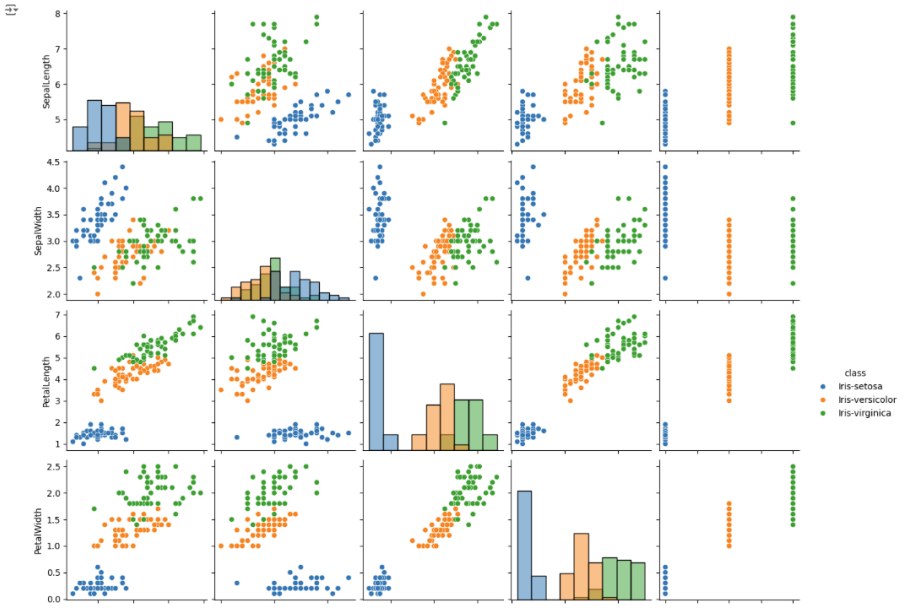
OUTPUT:

Support: 11

#Support Vector Classification graph

```
import seaborn as sns
sns.pairplot(df, hue='class',diag_kind='hist')
plt.show()
```

OUTPUT:



Result:

Thus implemented Support Vector Classification successfully and executed.

EX NO : 6 **DATE** :	**K-NEAREST NEIGHBOR CLASSIFICATION**

AIM:

To implement the K- Nearest Neighbor Classification.

PROCEDURE:

- Load the dataset containing features.
- Split the dataset into training and testing sets.
- Train a k-Neighbor Classifier model using the training data.
- Evaluate the model's performance using the testing data.
- Optionally, analyze confusion matrix and Classification Report their significance.

CODE:

#Importing libraries and dataset

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix , mean_squared_error
from sklearn.metrics import classification_report
```

#Reading Dataset

```
d=pd.read_csv('/content/drive/MyDrive/mlt/DiabetesClassify.csv')
d.head()
```

OUTPUT:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

#Splitting the dataset

```
x=d.iloc[:,:-1]
y=d.iloc[:,-1]
```

#Train test split

```
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.4,random_state=0)
x_train.head()
```

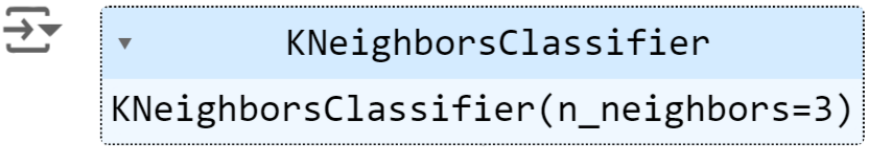
OUTPUT:

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age
60	2	84	0	0	0	0.0	0.304	21
618	9	112	82	24	0	28.2	1.282	50
346	1	139	46	19	83	28.7	0.654	22
294	0	161	50	0	0	21.9	0.254	65
231	6	134	80	37	370	46.2	0.238	46

#K-nearest neighbour

```
knn=KNeighborsClassifier(n_neighbors=3)
knn.fit(x_train,y_train)
```

OUTPUT:



#Predict

```
#Prediction
x_pred= knn.predict(x_train)
y_pred= knn.predict(x_test)
y_pred
```

OUTPUT:

```
array([1, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1,
       1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1,
       1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,
       0, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0,
       0, 1, 0, 1, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0,
       0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 1,
       0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1,
       0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1,
       1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0,
       0, 0, 0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0])
```

#KNN

```
for k in range(1,21):
        knn=KNeighborsClassifier(n_neighbors=k)
        knn.fit(x_train,y_train)
        ypred=knn.predict(x_test).astype(int)
        print("k=",k,"Accuracy score =", accuracy_score(ypred,y_test))
```

OUTPUT:

```
k= 1 Accuracy score = 0.6461038961038961
k= 2 Accuracy score = 0.7045454545454546
k= 3 Accuracy score = 0.7077922077922078
k= 4 Accuracy score = 0.737012987012987
k= 5 Accuracy score = 0.7435064935064936
k= 6 Accuracy score = 0.737012987012987
k= 7 Accuracy score = 0.75
k= 8 Accuracy score = 0.7532467532467533
k= 9 Accuracy score = 0.7727272727272727
k= 10 Accuracy score = 0.7467532467532467
k= 11 Accuracy score = 0.7532467532467533
k= 12 Accuracy score = 0.7467532467532467
k= 13 Accuracy score = 0.7467532467532467
k= 14 Accuracy score = 0.7467532467532467
k= 15 Accuracy score = 0.7435064935064936
k= 16 Accuracy score = 0.737012987012987
k= 17 Accuracy score = 0.7337662337662337
k= 18 Accuracy score = 0.7337662337662337
k= 19 Accuracy score = 0.737012987012987
k= 20 Accuracy score = 0.7435064935064936
```

#Accuracy score for x

```
acc=accuracy_score(x_pred,y_train)
print("Accuracy Score:",acc)
```

OUTPUT:

```
Accuracy Score: 0.8630434782608696
```

#Accuracy score for y

```
acc=accuracy_score(y_pred,y_test)
print("Accuracy Score:",acc)
```

OUTPUT:

```
Accuracy Score: 0.7077922077922078
```

#Confusion matrix

```
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n",cm)
```

OUTPUT:

```
Confusion Matrix:
 [[168  37]
 [ 53  50]]
```

#Mean squared error

```
mse = mean_squared_error(y_test, y_pred)
print("Mean Squared Error:", mse)
```

OUTPUT:

```
Mean Squared Error: 0.2922077922077922
```

#Classification report

```
print(classification_report(y_test, y_pred))
```

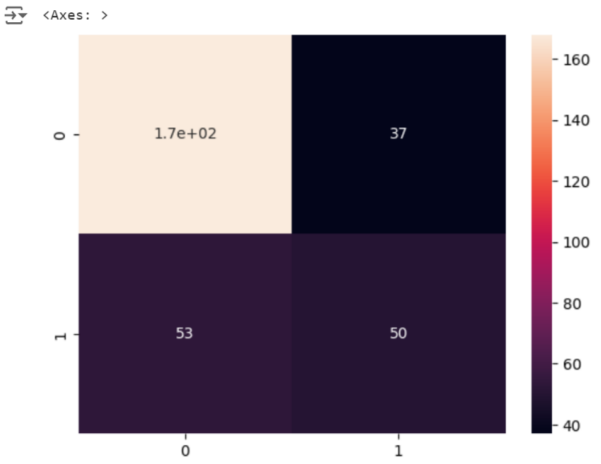
OUTPUT:

```
              precision    recall  f1-score   support

           0       0.76      0.82      0.79       205
           1       0.57      0.49      0.53       103

    accuracy                           0.71       308
   macro avg       0.67      0.65      0.66       308
weighted avg       0.70      0.71      0.70       308
```

#Heat map

```
import seaborn as sns
sns.heatmap(cm, annot=True)
```

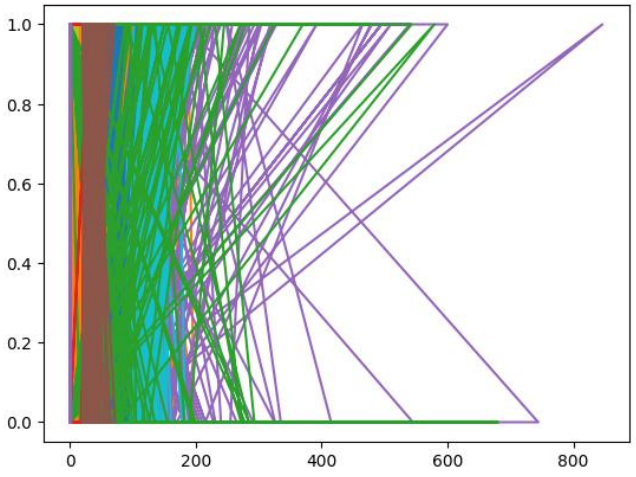
OUTPUT:



#Plotting

plt.plot(x_train,y_train)
plt.plot(x_test,y_test)

OUTPUT:



Result:

Thus implemented K-Nearest Neigbor Classification successfully and executed.

EX NO : 7A **DATE** :	**IMPLEMENTATION OF K MEANS CLUSTERING**

AIM:

To implement the K-Means Clustering.

PROCEDURE:

- Load the dataset containing features.
- Normalize the dataset using Min-Scaler Method.
- Train a K Means model.
- Evaluate the model's performance.
- Find the Inertia Score and Silhouette Score.
- Plot the graph for the Inertia score and Silhouette Score.

CODE:

#Importing libraries and dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import silhouette_score
from sklearn.cluster import KMeans
from sklearn.preprocessing import MinMaxScaler
import seaborn as sns
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

d=pd.read_csv('/content/drive/MyDrive/mlt/heart_data.csv')
d.head()
```

OUTPUT:

	age	chol
0	52	212
1	53	203
2	70	174
3	61	203
4	62	294

#Describing Dataset

```
d.describe()
```

OUTPUT:

	age	chol
count	1025.000000	1025.00000
mean	54.434146	246.00000
std	9.072290	51.59251
min	29.000000	126.00000
25%	48.000000	211.00000
50%	56.000000	240.00000
75%	61.000000	275.00000
max	77.000000	564.00000

#Checking for null Values

```
d.isna().sum()
```

OUTPUT:

	0
age	0
chol	0

dtype: int64

#Normalizing the dataset

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
# Transform the dataset
d['age']=scaler.fit_transform(d['age'].values.reshape(-1,1))
d['chol']=scaler.fit_transform(d['chol'].values.reshape(-1,1))
# Display the normalized data
d.head()
```

OUTPUT:

	age	chol
0	0.479167	0.196347
1	0.500000	0.175799
2	0.854167	0.109589
3	0.666667	0.175799
4	0.687500	0.383562

```
x_labels=model_labels[3]
x_labels
```

OUTPUT:

```
array([0, 0, 2, ..., 1, 0, 0], dtype=int32)
```

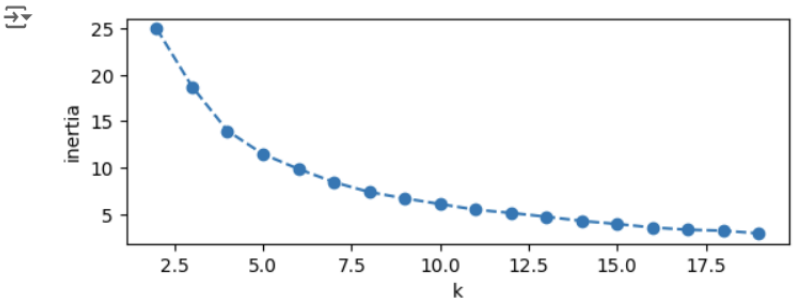
#Fitting KMeans Model

```
from sklearn.cluster import KMeans
model_labels={}
i_scores=[]
s_score=[]
centroids={}
for k in range(2,20):
  model=KMeans(n_clusters=k)
  model.fit(d)
  model_labels[k]=model.labels_
  i_scores.append(model.inertia_)
  s_score.append(silhouette_score(d,model.labels_))
  centroids[k]=model.cluster_centers_
```

 # Plot the inertia scores

```
plt.subplot(211)
plt.plot(range(2,20),i_scores,"o--")
plt.ylabel("inertia")
plt.xlabel("k")
plt.show()
```

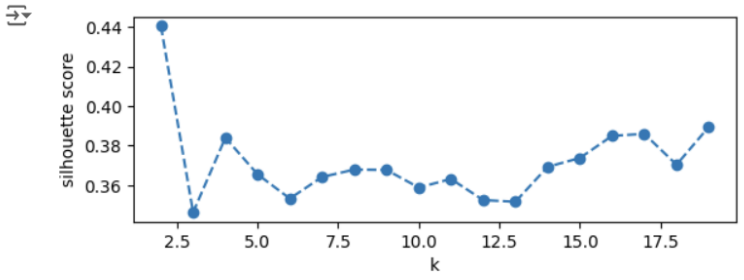
OUTPUT:



 # Plot the silhouette scores

```
plt.subplot(212)
plt.plot(range(2,20),s_score,"o--")
plt.xlabel("k")
plt.ylabel("silhouette score")
plt.show()
```

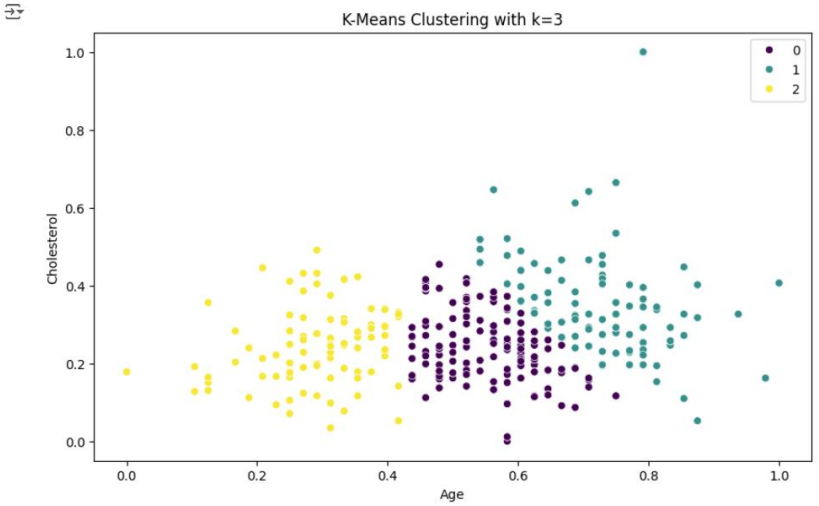
OUTPUT:



#K-Means Clustering Graph

```
k = 3
labels = model_labels[k]
plt.figure(figsize=(10, 6))
sns.scatterplot(x='age', y='chol', data=d, hue=labels, palette='viridis')
plt.title("K-Means Clustering with k=" + str(k))
plt.xlabel('Age')
plt.ylabel('Cholesterol')
plt.show()
```

OUTPUT:



Result:

Thus, implemented the K-Means Clustering Algorithm successfully and executed.

EX NO : 7B DATE :	IMPLEMENTATION OF HIERARCHICAL CLUSTERING

AIM:

To implement the Hierarchical Clustering.

PROCEDURE:

- Load the dataset containing features.
- Normalize the dataset using Min-Scaler Method.
- Train a K Means model.
- Evaluate the model's performance.
- Plot the graph for the Hierarchical Clustering.

CODE:

```
#Importing libraries and dataset
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
#Importing dataset
d=pd.read_csv('/content/drive/MyDrive/mlt/Mall_Customers.csv')
d = d.iloc[:,1:]
d.head()
```

OUTPUT:

	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	Male	19	15	39
1	Male	21	15	81
2	Female	20	16	6
3	Female	23	16	77
4	Female	31	17	40

```
d.isna().sum()
```

OUTPUT:

	0
Gender	0
Age	0
Annual Income (k$)	0
Spending Score (1-100)	0

dtype: int64

```
from sklearn import preprocessing
label_encoder = preprocessing.LabelEncoder()
d['Gender']= label_encoder.fit_transform(d['Gender'])
d.head()
```

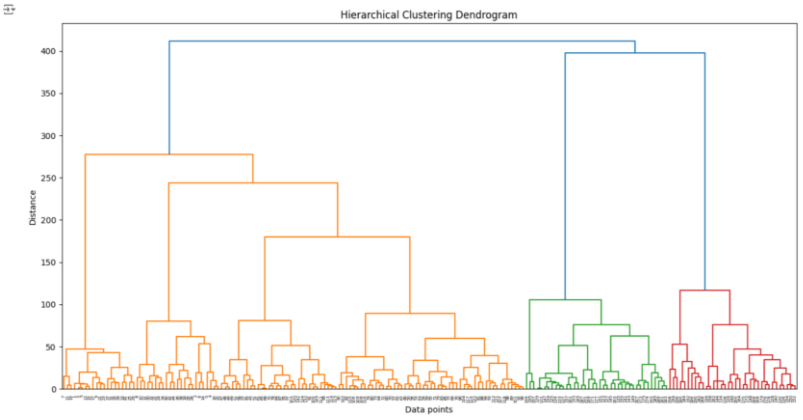
OUTPUT:

	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	19	15	39
1	1	21	15	81
2	0	20	16	6
3	0	23	16	77
4	0	31	17	40

#Dendogram

```
import matplotlib.pyplot as plt
import scipy.cluster.hierarchy as sch
plt.figure(1, figsize = (16,8))
plt.title('Hierarchical Clustering Dendrogram')
plt.xlabel('Data points')
plt.ylabel('Distance')
dendrogram = sch.dendrogram(sch.linkage(d, method = 'ward'))
```

OUTPUT:




```
from sklearn.cluster import AgglomerativeClustering
hc = AgglomerativeClustering(n_clusters = 5, linkage = 'average')
y_hc = hc.fit_predict(d)
y_hc
```

OUTPUT:

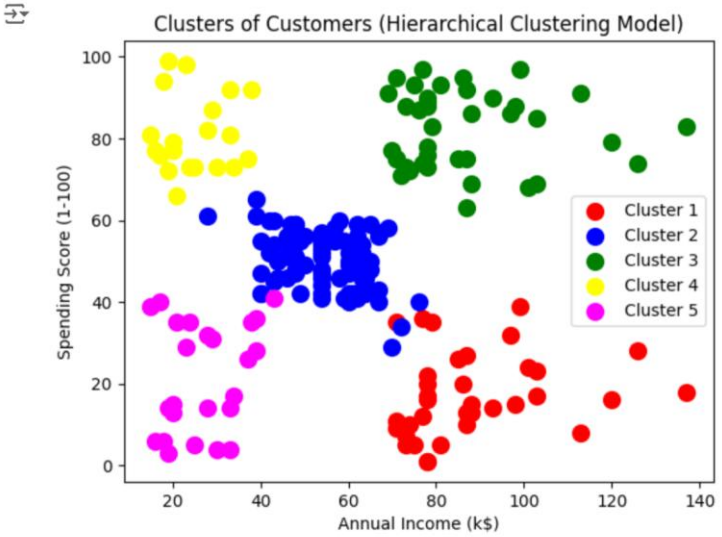
```
array([4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3,
       4, 3, 4, 3, 4, 1, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 3, 4, 1,
       4, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 4, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 1, 2, 0, 2, 0, 2, 0, 2,
       1, 2, 0, 2, 0, 2, 0, 2, 0, 2, 1, 2, 0, 2, 0, 2, 0, 2, 0, 2,
       0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2,
       0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2, 0, 2,
       0, 2])
```

#Hierarchical Clustering Model

```
X = d.iloc[ : ,[2,3]].values
plt.scatter(X[y_hc == 0, 0], X[y_hc == 0, 1], s = 100, c = 'red', label = 'Cluster 1')
plt.scatter(X[y_hc == 1, 0], X[y_hc == 1, 1], s = 100, c = 'blue', label = 'Cluster 2')
plt.scatter(X[y_hc == 2, 0], X[y_hc == 2, 1], s = 100, c = 'green', label = 'Cluster 3')
plt.scatter(X[y_hc == 3, 0], X[y_hc == 3, 1], s = 100, c = 'yellow', label = 'Cluster 4')
plt.scatter(X[y_hc == 4, 0], X[y_hc == 4, 1], s = 100, c = 'magenta', label = 'Cluster 5')
```

```
plt.title('Clusters of Customers (Hierarchical Clustering Model)')
plt.xlabel('Annual Income (k$)')
plt.ylabel('Spending Score (1-100)')
plt.legend()
plt.show()
```

OUTPUT:



Result:

Thus, implemented the Hierarchical Clustering successfully and executed.

EX NO : 8A **DATE** :	**IMPLEMENTATION OF APRIORI CLASSIFICATION**

AIM:

To implement the Apriori Clustering.

PROCEDURE:

- Load the dataset containing features.
- Fit the dataset using Transaction Encoder.
- Store the items combination in the list.
- Identify support and confidence.
- Display the table with support, lift, leverage, conviction.

CODE:

#Installing mlxend

```
!pip install mlxtend
```

OUTPUT:

```
/usr/local/lib/python3.10/dist-packages/ipykernel/ipkernel.py:283: DeprecationWarning: `should_run_async` will not call `transform_cell` automatically in the future. Please pass the result to `transformed_cell
  and should_run_async(code)
Requirement already satisfied: mlxtend in /usr/local/lib/python3.10/dist-packages (0.23.1)
Requirement already satisfied: scipy>=1.2.1 in /usr/local/lib/python3.10/dist-packages (from mlxtend) (1.13.1)
Requirement already satisfied: numpy>=1.16.2 in /usr/local/lib/python3.10/dist-packages (from mlxtend) (1.26.4)
Requirement already satisfied: pandas>=0.24.2 in /usr/local/lib/python3.10/dist-packages (from mlxtend) (2.2.2)
Requirement already satisfied: scikit-learn>=1.0.2 in /usr/local/lib/python3.10/dist-packages (from mlxtend) (1.5.2)
Requirement already satisfied: matplotlib>=3.0.0 in /usr/local/lib/python3.10/dist-packages (from mlxtend) (3.7.1)
Requirement already satisfied: joblib>=0.13.2 in /usr/local/lib/python3.10/dist-packages (from mlxtend) (1.4.2)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (1.3.0)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (4.54.1)
Requirement already satisfied: kiwisolver>=1.0.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (1.4.7)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (24.1)
Requirement already satisfied: pillow>=6.2.0 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (10.4.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.10/dist-packages (from matplotlib>=3.0.0->mlxtend) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.24.2->mlxtend) (2024.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.10/dist-packages (from pandas>=0.24.2->mlxtend) (2024.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.10/dist-packages (from scikit-learn>=1.0.2->mlxtend) (3.5.0)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.10/dist-packages (from python-dateutil>=2.7->matplotlib>=3.0.0->mlxtend) (1.16.0)
```

#Importing Libraries

```
import pandas as pd
from mlxtend.preprocessing import TransactionEncoder
from mlxtend.frequent_patterns import apriori
from mlxtend.frequent_patterns import association_rules
pd.set_option('display.max_columns',None)
pd.set_option('display.max_rows',None)
data=pd.read_csv('/content/drive/MyDrive/GroceryStoreDataSet.csv', header=None)

data.head()
```

OUTPUT:

	0
0	MILK,BREAD,BISCUIT
1	BREAD,MILK,BISCUIT,CORNFLAKES
2	BREAD,TEA,BOURNVITA
3	JAM,MAGGI,BREAD,MILK
4	MAGGI,TEA,BISCUIT

#Information of the dataset

```
print(data.info())
data=list(data[0].apply(lambda x:x.split(',')))
```

OUTPUT:

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20 entries, 0 to 19
Data columns (total 1 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   0       20 non-null     object
dtypes: object(1)
memory usage: 288.0+ bytes
None
```

#Printing the data stored in the list data

data

OUTPUT:

```
[['MILK', 'BREAD', 'BISCUIT'],
 ['BREAD', 'MILK', 'BISCUIT', 'CORNFLAKES'],
 ['BREAD', 'TEA', 'BOURNVITA'],
 ['JAM', 'MAGGI', 'BREAD', 'MILK'],
 ['MAGGI', 'TEA', 'BISCUIT'],
 ['BREAD', 'TEA', 'BOURNVITA'],
 ['MAGGI', 'TEA', 'CORNFLAKES'],
 ['MAGGI', 'BREAD', 'TEA', 'BISCUIT'],
 ['JAM', 'MAGGI', 'BREAD', 'TEA'],
 ['BREAD', 'MILK'],
 ['COFFEE', 'COCK', 'BISCUIT', 'CORNFLAKES'],
 ['COFFEE', 'COCK', 'BISCUIT', 'CORNFLAKES'],
 ['COFFEE', 'SUGER', 'BOURNVITA'],
 ['BREAD', 'COFFEE', 'COCK'],
 ['BREAD', 'SUGER', 'BISCUIT'],
 ['COFFEE', 'SUGER', 'CORNFLAKES'],
 ['BREAD', 'SUGER', 'BOURNVITA'],
 ['BREAD', 'COFFEE', 'SUGER'],
 ['BREAD', 'COFFEE', 'SUGER'],
 ['TEA', 'MILK', 'COFFEE', 'CORNFLAKES']]
```

#fit and transform data using Transaction Encoder

```
te = TransactionEncoder()
te_ary = te.fit(data).transform(data)
df = pd.DataFrame(te_ary, columns=te.columns_)
df
```

OUTPUT:

	BISCUIT	BOURNVITA	BREAD	COCK	COFFEE	CORNFLAKES	JAM	MAGGI	MILK	SUGER	TEA
0	True	False	True	False	False	False	False	False	True	False	False
1	True	False	True	False	False	True	False	False	True	False	False
2	False	True	True	False	False	False	False	False	False	False	True
3	False	False	True	False	False	False	True	True	True	False	False
4	True	False	False	False	False	False	False	True	False	False	True
5	False	True	True	False	False	False	False	False	False	False	True
6	False	False	False	False	False	True	False	True	False	False	True
7	True	False	True	False	False	False	False	True	False	False	True
8	False	False	True	False	False	False	True	True	False	False	True
9	False	False	True	False	False	False	False	False	True	False	False
10	True	False	False	True	True	True	False	False	False	False	False
11	True	False	False	True	True	True	False	False	False	False	False
12	False	True	False	False	True	False	False	False	False	True	False
13	False	False	True	True	True	False	False	False	False	False	False
14	True	False	True	False	False	False	False	False	False	True	False
15	False	False	False	False	True	True	False	False	False	True	False
16	False	True	True	False	False	False	False	False	False	True	False
17	False	False	True	False	True	False	False	False	False	True	False
18	False	False	True	False	True	False	False	False	False	True	False
19	False	False	False	False	True	True	False	False	True	False	True

#Support and itemsets

```
df1=apriori(df,min_support=0.01,use_colnames=True)
print(df1)
print(df1.sort_values(by="support",ascending=False))
df_ar=association_rules(df1,metric="confidence",min_threshold=0.5)
```

OUTPUT:

```
    support              itemsets
0      0.35             (BISCUIT)
1      0.20            (BOURNVITA)
2      0.65               (BREAD)
3      0.15                (COCK)
4      0.40              (COFFEE)
5      0.30          (CORNFLAKES)
6      0.10                 (JAM)
7      0.25               (MAGGI)
8      0.25                (MILK)
9      0.30               (SUGER)
10     0.35                 (TEA)
11     0.20       (BREAD, BISCUIT)
12     0.10        (COCK, BISCUIT)
13     0.10      (COFFEE, BISCUIT)
14     0.15  (CORNFLAKES, BISCUIT)
```

df_ar.head(8)

	antecedents	consequents	antecedent support	consequent support	support	confidence	lift	leverage	conviction	zhangs_metric
0	(BISCUIT)	(BREAD)	0.35	0.65	0.20	0.571429	0.879121	-0.0275	0.816667	-0.174603
1	(COCK)	(BISCUIT)	0.15	0.35	0.10	0.666667	1.904762	0.0475	1.950000	0.558824
2	(CORNFLAKES)	(BISCUIT)	0.30	0.35	0.15	0.500000	1.428571	0.0450	1.300000	0.428571
3	(BOURNVITA)	(BREAD)	0.20	0.65	0.15	0.750000	1.153846	0.0200	1.400000	0.166667
4	(BOURNVITA)	(SUGER)	0.20	0.30	0.10	0.500000	1.666667	0.0400	1.400000	0.500000
5	(BOURNVITA)	(TEA)	0.20	0.35	0.10	0.500000	1.428571	0.0300	1.300000	0.375000
6	(JAM)	(BREAD)	0.10	0.65	0.10	1.000000	1.538462	0.0350	inf	0.388889
7	(MAGGI)	(BREAD)	0.25	0.65	0.15	0.600000	0.923077	-0.0125	0.875000	-0.100000

Result:

Thus, implemented the Apriori Clustering successfully and executed.

EX NO : 8B DATE :	IMPLEMENTATION OF ECLAT ASSOCIATION

AIM:

To implement the ECLAT Association.

PROCEDURE:

- Load the dataset containing features.
- Fit the dataset using Transaction Encoder.
- Store the items combination in the list.
- Identify support and confidence.
- Display the table with support and itemset.

CODE:

#Importing necessary libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

#Loading the dataset

```
df = pd.read_csv('/content/drive/MyDrive/Data/Market_Basket_Optimisation (1).csv',header=None)
```

#Displaying the Dataset

```
df.head()
```

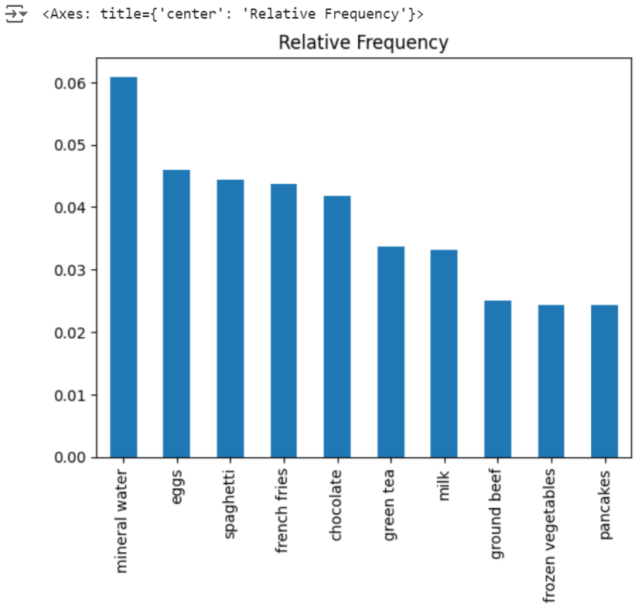
OUTPUT:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	shrimp	almonds	avocado	vegetables mix	green grapes	whole weat flour	yams	cottage cheese	energy drink	tomato juice	low fat yogurt	green tea	honey	salad	mineral water	salmon	antioxydant juice	frozen smoothie	spinach	olive oil
1	burgers	meatballs	eggs	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	chutney	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	turkey	avocado	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	mineral water	milk	energy bar	whole wheat rice	green tea	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

#Graph of the itemset

```
df.stack().value_counts(normalize=True)[:10].plot(kind='bar', title='Relative Frequency')
```

OUTPUT:



```
transactions=[]
for i in range(df.shape[0]):
  row=df.iloc[i].dropna().tolist()
  transactions.append(row)
transactions[:10]
```

OUTPUT:

```
[['shrimp',
  'almonds',
  'avocado',
  'vegetables mix',
  'green grapes',
  'whole weat flour',
  'yams',
  'cottage cheese',
  'energy drink',
  'tomato juice',
  'low fat yogurt',
  'green tea',
  'honey',
  'salad',
  'mineral water',
  'salmon',
  'antioxydant juice',
  'frozen smoothie',
  'spinach',
  'olive oil'],
 ['burgers', 'meatballs', 'eggs'],
 ['chutney'],
 ['turkey', 'avocado'],
 ['mineral water', 'milk', 'energy bar', 'whole wheat rice', 'green tea'],
 ['low fat yogurt'],
 ['whole wheat pasta', 'french fries'],
 ['soup', 'light cream', 'shallot'],
 ['frozen vegetables', 'spaghetti', 'green tea'],
 ['french fries']]
```

```
df.head()
```

OUTPUT:

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
0	shrimp	almonds	avocado	vegetables mix	green grapes	whole weat flour	yams	cottage cheese	energy drink	tomato juice	low fat yogurt	green tea	honey	salad	mineral water	salmon	antioxydant juice	frozen smoothie	spinach	olive oil
1	burgers	meatballs	eggs	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
2	chutney	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
3	turkey	avocado	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN
4	mineral water	milk	energy bar	whole wheat rice	green tea	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN

```
!pip install apyori
```

OUTPUT:

```
Collecting apyori
  Downloading apyori-1.1.2.tar.gz (8.6 kB)
  Preparing metadata (setup.py) ... done
Building wheels for collected packages: apyori
  Building wheel for apyori (setup.py) ... done
  Created wheel for apyori: filename=apyori-1.1.2-py3-none-any.whl size=5954 sha256=605bb5ae0e9d1bfc2439347d1651d6b3357622a73853201007cce51e5cd8ebfd
  Stored in directory: /root/.cache/pip/wheels/c4/1a/79/20f55c470a50bb3702a8cb7c94d8ada15573538c7f4baebe2d
Successfully built apyori
Installing collected packages: apyori
Successfully installed apyori-1.1.2
```

#Importing Apriori

```
from apyori import apriori
rules=apriori(transactions=transactions,min_support=0.004,min_length=2,max_length=0)
results=list(rules)
def inspect(results):
  item_sets=[]
  supports=[]
  for result in results:
    for subet in result[2]:
      item_sets.append(tuple(result[0]))
      supports.append(result[1])
```

```
  return list(zip(item_sets,supports))
results_df=pd.DataFrame(inspect(results),columns=['Item Set','Support'])
results_df.shape
```

OUTPUT:

```
(3781, 2)
```

```
pd.set_option('display.max_rows', df.shape[0])
results_df.sort_values('Support', ascending=False)[:100]
```

OUTPUT:

	Item Set	Support
69	(mineral water,)	0.238368
34	(eggs,)	0.179709
96	(spaghetti,)	0.174110
40	(french fries,)	0.170911
23	(chocolate,)	0.163845
51	(green tea,)	0.132116
68	(milk,)	0.129583
52	(ground beef,)	0.098254
46	(frozen vegetables,)	0.095321
78	(pancakes,)	0.095054
13	(burgers,)	0.087188
15	(cake,)	0.081056

Result:

Thus, implemented the ECLAT Association successfully and the output is verified..

EX NO : 9A **DATE** :	**IMPLEMENTATION OF PRINCIPAL COMPONENT ANALYSIS (PCA)**

AIM:

To implement the Principal Component Analysis (PCA).

PROCEDURE:

- Load the dataset containing features.
- Scale the feature using StandardScaler.
- Fit the dataset using PCA.
- Split the dataset using Logistic Classification model.
- Plot the graph using scatter plot.
- Find the classification matrix and Confusion Matrix

CODE:

#Importing necessary Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
```

#Loading dataset

```
data=load_iris()
X=data['data']
y=data['target']
```

#Displaying data of the dataset

```
X
```

OUTPUT:

```
array([[5.1, 3.5, 1.4, 0.2],
       [4.9, 3. , 1.4, 0.2],
       [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2],
       [5. , 3.6, 1.4, 0.2],
       [5.4, 3.9, 1.7, 0.4],
       [4.6, 3.4, 1.4, 0.3],
       [5. , 3.4, 1.5, 0.2],
       [4.4, 2.9, 1.4, 0.2],
       [4.9, 3.1, 1.5, 0.1],
       [5.4, 3.7, 1.5, 0.2],
       [4.8, 3.4, 1.6, 0.2],
       [4.8, 3. , 1.4, 0.1],
       [4.3, 3. , 1.1, 0.1],
       [5.8, 4. , 1.2, 0.2],
```

#Displaying target of the dataset

```
y
```

OUTPUT:

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

#Multiclass logistic regression #PCA

```
from sklearn.decomposition import PCA
pca=PCA(n_components=2)
x_pca=pca.fit_transform(X)
```

#Spliting dataset into train and test.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(x_pca, y, test_size = 0.20, random_state = 82)
```

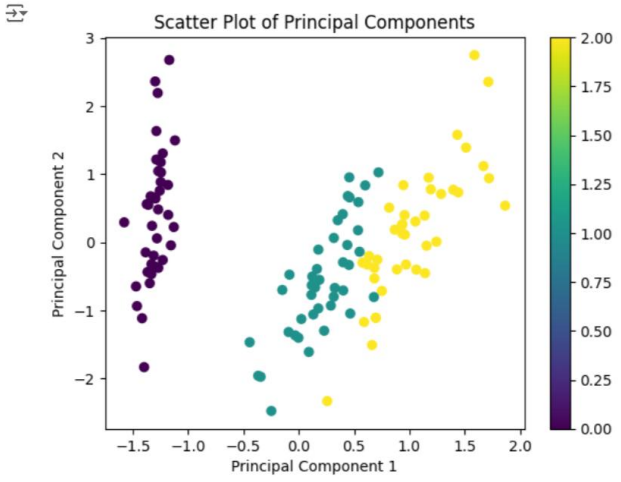
 # Feature Scaling to bring the variable in a single scale

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

#Scatter plot

```
plt.scatter(X_train[:,0], X_train[:,1], c=y_train)
plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("Scatter Plot of Principal Components")
plt.colorbar()
plt.show()
```

OUTPUT:



 # Fitting Multiclass Logistic Classification to the Training set

```
from sklearn.linear_model import LogisticRegression
logisticregression = LogisticRegression()
logisticregression.fit(X_train, y_train)
```

OUTPUT:

```
LogisticRegression
LogisticRegression()
```

y_pred = logisticregression.predict(X_test)
y_pred

OUTPUT:

```
array([2, 2, 0, 0, 0, 2, 1, 1, 2, 2, 1, 2, 0, 0, 0, 0, 2, 1, 0, 1, 0, 2,
       0, 2, 2, 1, 2, 0, 2, 1])
```

#Confusion Matrix

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

OUTPUT:

```
[[11  0  0]
 [ 0  7  2]
 [ 0  0 10]]
```

#Classification_report

```
from sklearn.metrics import classification_report as cr
print(cr(y_test,y_pred))
```

OUTPUT:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11
1	1.00	0.78	0.88	9
2	0.83	1.00	0.91	10
accuracy			0.93	30
macro avg	0.94	0.93	0.93	30
weighted avg	0.94	0.93	0.93	30

Result:

Thus, implemented the Principal Component Analysis successfully and executed.

EX NO : 9B **DATE** :	**IMPLEMENTATION OF LINEAR DISCRIMINANT ANALYSIS (LDA)**

AIM:

To implement the Linear Discriminant Analysis(LDA).

PROCEDURE:

- Load the dataset containing features.
- Scale the feature using StandardScaler.
- Fit the dataset using LDA.
- Split the dataset using Logistic Classification model.
- Plot the graph using scatter plot.
- Find the classification matrix and Confusion Matrix

CODE:

#Importing Necessary Libraries

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_iris
```

#Importing dataset

```
data=load_iris()
X=data['data']
y=data['target']
```

#Displaying data of the dataset

```
X
```

OUTPUT:

```
array([[5.1, 3.5, 1.4, 0.2],
       [4.9, 3. , 1.4, 0.2],
       [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2],
       [5. , 3.6, 1.4, 0.2],
       [5.4, 3.9, 1.7, 0.4],
       [4.6, 3.4, 1.4, 0.3],
       [5. , 3.4, 1.5, 0.2],
       [4.4, 2.9, 1.4, 0.2],
       [4.9, 3.1, 1.5, 0.1],
       [5.4, 3.7, 1.5, 0.2],
       [4.8, 3.4, 1.6, 0.2],
       [4.8, 3. , 1.4, 0.1],
       [4.3, 3. , 1.1, 0.1],
       [5.8, 4. , 1.2, 0.2],
```

#Displaying target of the dataset

```
y
```

OUTPUT:

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2])
```

#fitting LDA model

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
lda=LDA(n_components=2)
x_lda=lda.fit_transform(X,y)
```

#Splitting dataset into train and test.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(x_lda, y, test_size = 0.20, random_state = 82)
```

 # Feature Scaling to bring the variable in a single scale

```
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)
```

 # Fitting Multiclass Logistic Classification to the Training set

```
from sklearn.linear_model import LogisticRegression
logisticregression = LogisticRegression()
logisticregression.fit(X_train, y_train)
```

OUTPUT:

```
LogisticRegression
LogisticRegression()
```

 # Predicting the Test set results

```
y_pred = logisticregression.predict(X_test)
print(y_pred)
```

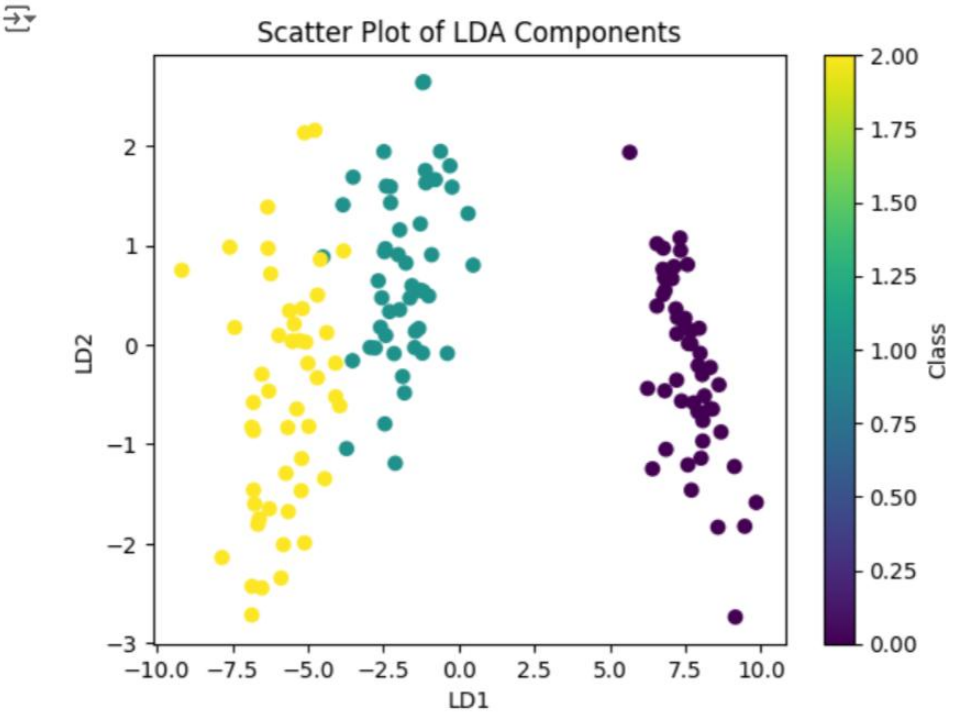
OUTPUT:

```
[2 2 0 0 0 2 1 1 1 1 1 0 0 0 0 2 1 0 1 0 2 0 2 2 1 2 0 2 1]
```

#scatter plot

```
# Assuming x_lda contains the transformed data with 2 components
plt.scatter(x_lda[:, 0], x_lda[:, 1], c=y)
plt.xlabel('LD1')
plt.ylabel('LD2')
plt.title('Scatter Plot of LDA Components')
plt.colorbar(label='Class')
plt.show()
```

OUTPUT:



 # Confusion Matrix

```
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(y_test, y_pred)
print(cm)
```

OUTPUT:

```
[[11  0  0]
 [ 0  8  1]
 [ 0  2  8]]
```

#Classification_Report

```
from sklearn.metrics import classification_report as cr
print(cr(y_test,y_pred))
```

OUTPUT:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	11
1	0.80	0.89	0.84	9
2	0.89	0.80	0.84	10
accuracy			0.90	30
macro avg	0.90	0.90	0.89	30
weighted avg	0.90	0.90	0.90	30

Result:

Thus, implemented the Linear Discriminant Analysis successfully and executed.

EX NO : 10A **DATE** :	**IMPLEMENTATION OF SINGLE LAYER PERCEPTRON**

AIM:

To implement the Single Layer Perceptron.

PROCEDURE:

- Load the dataset containing features.
- Import the necessary Libraries.
- Fit the dataset using Perceptron
- Split the dataset.
- Find the accuracy of the model.

CODE:

```
#Importing necessary Libraries
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import Perceptron
from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn.metrics import accuracy_score
from sklearn.metrics import confusion_matrix

# Load the Breast Cancer dataset
data = load_breast_cancer()
X = data.data
y = data.target

#Displaying x values
X
```

OUTPUT:

```
array([[1.799e+01, 1.038e+01, 1.228e+02, ..., 2.654e-01, 4.601e-01,
        1.189e-01],
       [2.057e+01, 1.777e+01, 1.329e+02, ..., 1.860e-01, 2.750e-01,
        8.902e-02],
       [1.969e+01, 2.125e+01, 1.300e+02, ..., 2.430e-01, 3.613e-01,
        8.758e-02],
       ...,
       [1.660e+01, 2.808e+01, 1.083e+02, ..., 1.418e-01, 2.218e-01,
        7.820e-02],
       [2.060e+01, 2.933e+01, 1.401e+02, ..., 2.650e-01, 4.087e-01,
        1.240e-01],
       [7.760e+00, 2.454e+01, 4.792e+01, ..., 0.000e+00, 2.871e-01,
        7.039e-02]])
```

#Displaying Y values

OUTPUT:

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0,
       1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0,
       1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
       1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1,
       1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0,
       0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0,
       1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 1, 0, 0, 1, 0, 1, 1,
       1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0,
       0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0,
       0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0,
       1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 1, 1,
       1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1,
       1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0,
       1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1,
       1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1,
       1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1])
```

#Shape of the x and y

```
print(X.shape)
print(y.shape)
```

OUTPUT:

```
(569, 30)
(569,)
```

 # Split the dataset into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

OUTPUT:

```
(398, 30)
(171, 30)
(398,)
(171,)
```

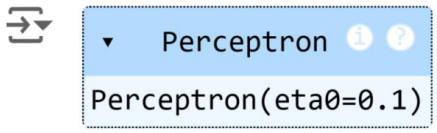
 # Create a Perceptron model

```
clf = Perceptron(max_iter=1000, eta0=0.1)
```

 # Train the model

```
clf.fit(X_train, y_train)
```

OUTPUT:



 # Make predictions

```
y_pred = clf.predict(X_test)
print(y_pred)
```

OUTPUT:

```
[0 0 0 1 1 0 0 0 1 1 1 0 1 0 1 0 1 1 1 0 0 1 0 1 1 1 1 1 1 0 1 1 1 0 1 1 0
 1 0 1 1 0 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 0 1 1 1 0 0 1 1 1 0 0 1 1 0 0 1 1
 1 1 1 0 1 1 0 1 0 0 0 0 0 0 0 1 1 1 1 0 1 1 1 0 0 1 0 0 1 0 0 1 1 1 0 1 0 0
 1 1 0 1 0 1 1 1 0 0 1 1 0 1 0 0 1 1 0 0 1 0 1 0 0 1 1 0 0 1 0 1 1 0 1 0 0
 0 1 0 1 1 1 1 0 0 1 1 1 1 1 1 1 0 1 1 1 1 0 1]
```

 # Evaluate accuracy

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

OUTPUT:

```
Accuracy: 0.935672514619883
```

Result:

Thus, implemented the Single Layer Perceptron successfully and executed.

EX NO : 10B **DATE** :	**IMPLEMENTATION OF MULTI LAYER PERCEPTRON**

AIM:

To implement the Multi Layer Perceptron.

PROCEDURE:

- Load the dataset containing features.
- Import the necessary Libraries.
- Scale the feature using StandardScaler.
- Fit the dataset using Multi Layer Perceptron.
- Split the dataset.
- Find the accuracy of the model.

CODE:

#Importing necessary Libraries

```
from sklearn.neural_network import MLPClassifier
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

 # Load the Breast Cancer dataset

```
data = load_breast_cancer()
X = data.data
y = data.target
```

#Displaying x values

```
X
```

OUTPUT:

```
array([[1.799e+01, 1.038e+01, 1.228e+02, ..., 2.654e-01, 4.601e-01,
        1.189e-01],
       [2.057e+01, 1.777e+01, 1.329e+02, ..., 1.860e-01, 2.750e-01,
        8.902e-02],
       [1.969e+01, 2.125e+01, 1.300e+02, ..., 2.430e-01, 3.613e-01,
        8.758e-02],
       ...,
       [1.660e+01, 2.808e+01, 1.083e+02, ..., 1.418e-01, 2.218e-01,
        7.820e-02],
       [2.060e+01, 2.933e+01, 1.401e+02, ..., 2.650e-01, 4.087e-01,
        1.240e-01],
       [7.760e+00, 2.454e+01, 4.792e+01, ..., 0.000e+00, 2.871e-01,
        7.039e-02]])
```

#Displaying Y values

```
y
```

OUTPUT:

```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0,
       1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0,
       1, 1, 1, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 1,
       1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0, 0, 1, 0, 1, 0,
       0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 1,
       1, 0, 1, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 0, 0,
       0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0,
       1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1,
       1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 0, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0,
       0, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0,
       0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0,
       1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1, 1,
       1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 0, 1, 1, 0,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 0, 1, 1, 1, 1,
       1, 0, 1, 1, 0, 1, 0, 1, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0,
       1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 1, 1, 1, 1,
       1, 1, 1, 0, 1, 0, 1, 1, 0, 1, 1, 1, 1, 1, 0, 0, 1, 0, 1, 0, 1, 1,
       1, 1, 1, 0, 1, 1, 0, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1])
```

#Shape of the x and y

```
print(X.shape)
print(y.shape)
```

OUTPUT:

```
(569, 30)
(569,)
```

 # Split the dataset into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(X_train.shape)
print(X_test.shape)
print(y_train.shape)
print(y_test.shape)
```

OUTPUT:

```
(455, 30)
(114, 30)
(455,)
(114,)
```

 # Standardize features by removing the mean and scaling to unit variance

```
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

 # Create an MLPClassifier model

```
mlp = MLPClassifier(hidden_layer_sizes=(64, 32),
                    max_iter=1000, random_state=42)
```

 # Train the model

```
mlp.fit(X_train, y_train)
```

OUTPUT:

```
MLPClassifier
MLPClassifier(hidden_layer_sizes=(64, 32), max_iter=1000, random_state=42)
```

 # Make predictions on the test data

```
y_pred = mlp.predict(X_test)
print(y_pred)
```

OUTPUT:

```
[1 0 0 1 1 0 0 0 1 1 1 0 1 0 1 0 1 1 1 0 1 1 0 1 1 1 1 1 1 1 0 1 1 1 1 1 1 0
 1 0 1 1 0 1 1 1 1 1 1 1 1 1 0 0 1 1 1 1 1 0 0 1 1 0 0 1 1 1 0 0 1 1 0 0 1 0
 1 1 1 1 1 1 0 1 0 0 0 0 0 0 1 1 1 0 1 1 1 1 0 0 1 0 0 1 0 0 1 1 1 0 1 1 0
 1 1 0]
```

 # Evaluate accuracy

```
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)
```

OUTPUT:

```
Accuracy: 0.9736842105263158
```

#Confusion Matrix

```
confusion = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(confusion)
```

OUTPUT:

```
Confusion Matrix:
[[41  2]
 [ 1 70]]
```

#Classification Report

```
classification_report = classification_report(y_test, y_pred)
print("Classification Report:")
print(classification_report)
```

OUTPUT:

```
Classification Report:
              precision    recall  f1-score   support

           0       0.98      0.95      0.96        43
           1       0.97      0.99      0.98        71

    accuracy                           0.97       114
   macro avg       0.97      0.97      0.97       114
weighted avg       0.97      0.97      0.97       114
```

Result:

Thus, implemented the Multi-Layer Perceptron successfully and executed.